UML Class Diagram for Robotic Path Planning

Bruun, E. Flores, J. Kumar, V.

December 2, 2019

Input

Rectangle

+ dim: int
+ bottom_left: Point
+ top_right: Point
+ shape_name: string

+ __init__(Point p1, Point p2)
+ inside(Point p)
+ intersects(Point p1, Point p2)

Circle

+ dim: int
+ radius: double
+ center: Point
+ shape_name: string

+ __init__(double radius, Point center)
+ inside(Point p)
+ intersects(Point p1, Point p2)

Shape

+ dim: int

+ is_point_inside()

Domain

+ dim: int
+ number_of_points: int
+ Obstacles: Obstacle
+ Origin: Point
+ Goal: Point

+ is_point_free(Point p): boolean
+ refine(double step_size):
+ update_vertex_list()

Obstacle

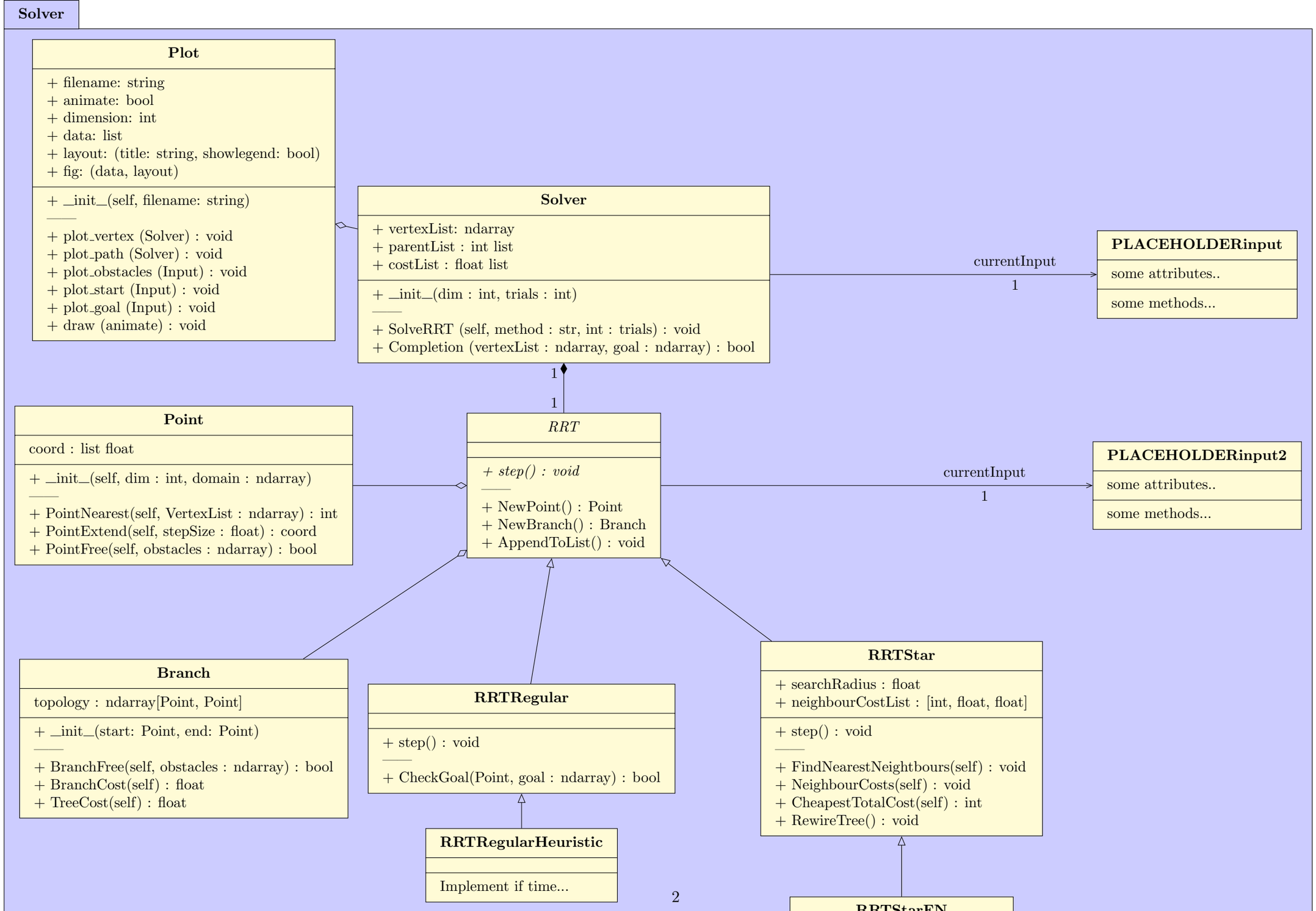
+ dim: int
+ shape_name: string
+

+ __init__(shape_name)

RRTParameters

+ file_name: string

__init__()
+ parse_parameters(string file_name)
+ void declare_entry(string entry_name,
string default_value,
Reg_Ex(?) pattern)
+ set(string entry_name, string new_value)
+ print_paramters()


Solver
Plot
+ filename: string
+ animate: bool
+ dimension: int
+ data: list
+ layout: (title: string, showlegend: bool)
+ fig: (data, layout)
+ __init__(self, filename: string)
+ plot_vertex (Solver) : void
+ plot_path (Solver) : void
+ plot_obstacles (Input) : void
+ plot_start (Input) : void
+ plot_goal (Input) : void
+ draw (animate) : void
Solver
+ vertexList: ndarray
+ parentList : int list
+ costList : float list
+ __init__(dim : int, trials : int)
+ SolveRRT (self, method : str, int : trials) : void
+ Completion (vertexList : ndarray, goal : ndarray) : bool
currentInput
1
PLACEHOLDERinput
some attributes..
some methods...
1
1
Point
coord : list float
+ __init__(self, dim : int, domain : ndarray)
+ PointNearest(self, VertexList : ndarray) : int
+ PointExtend(self, stepSize : float) : coord
+ PointFree(self, obstacles : ndarray) : bool
RRT
+ step() : void
+ NewPoint() : Point
+ NewBranch() : Branch
+ AppendToList() : void
currentInput
1
PLACEHOLDERinput2
some attributes..
some methods...
Branch
topology : ndarray[Point, Point]
+ __init__(start: Point, end: Point)
+ BranchFree(self, obstacles : ndarray) : bool
+ BranchCost(self) : float
+ TreeCost(self) : float
RRTRegular
+ step() : void
+ CheckGoal(Point, goal : ndarray) : bool
RRTStar
+ searchRadius : float
+ neighbourCostList : [int, float, float]
+ step() : void
+ FindNearestNeightbours(self) : void
+ NeighbourCosts(self) : void
+ CheapestTotalCost(self) : int
+ RewireTree() : void
RRTRegularHeuristic
Implement if time...
RRTStarFN